



本科生毕业论文（设计）

题目： 密码学中布谷鸟哈希算法的
实现与分析

姓 名 赵宇涵

学 号 19335279

院 系 计算机学院

专 业 网络空间安全

指导教师 田海博 副教授

2024 年 5 月 10 日

密码学中布谷鸟哈希算法的 实现与分析

Implementation and Analysis of Cuckoo Hash Algorithms in Cryptography

姓 名 赵宇涵

学 号 19335279

院 系 计算机学院

专 业 网络空间安全

指导教师 田海博 副教授

2024 年 5 月 10 日

学术诚信声明

本人郑重声明：所呈交的毕业论文（设计），是本人在导师的指导下，独立进行研究工作所取得的成果。除文中已经注明引用的内容外，本论文（设计）不包含任何其他个人或集体已经发表或撰写过的作品成果。对本论文（设计）的研究做出重要贡献的个人和集体，均已在文中以明确方式标明。本论文（设计）的知识产权归属于培养单位。本人完全意识到本声明的法律结果由本人承担。

作者签名：

日 期： 年 月 日

摘要

近年来，随着网络的建设与发展，对海量信息的加密与存储的需求日益增长。计算机服务器时常要访问庞大容量的数据库并完成预期的操作。依赖于高性能的哈希函数，哈希表形式的数据结构能够实现对数据的快速存储、高效访问。目前，在密码学领域，SHA-2 系列函数的加密功能可靠性突出，具有得天独厚的优势；布谷鸟哈希表的结构具有很强的查询性能，空间利用率也很高，但存在插入过程中遇到死循环导致内存开销变大，插入总时间变长的问题。

文中提出了以 SHA-256 散列函数为基础设计哈希函数，以布谷鸟哈希算法填充哈希表，并增加回收碰撞死循环键值对的额外链表，构建出的简单布谷鸟哈希电话号码管理系统。在消除插入过程出现死循环导致再哈希造成的时间空间消耗条件下，更大程度上发挥布谷鸟哈希算法负载均衡的优点。

本文通过采集多组电话号码数据集进行实验，最终观察在预设哈希表的空间是数据集项数的不同倍率下，出现死循环的概率，以及哈希表的空间性能，插入性能，查询和删除性能，并对其进行总结和分析。

关键词：负载均衡，SHA-2，布谷鸟哈希算法，密码学

ABSTRACT

In recent years, with the construction and development of networks, the demand for encryption and storage of massive information has been increasing. Computer servers often need to access large databases and complete expected operations. Relying on high-performance hash functions, hash table data structures can achieve fast storage and efficient access to data. At present, in the field of cryptography, the SHA-2 series encryption functions have excellent reliability and unique advantages; The structure of the cuckoo hash table has strong query performance and high space utilization, but there is a problem of encountering dead loops during insertion, resulting in increased memory overhead and longer total insertion time.

The article proposes a simple cuckoo hash phone number management system based on the SHA-256 hash function, using the cuckoo hash algorithm to fill the hash table, and adding an additional linked list to collect collision dead loop key value pairs. Under the condition of eliminating the time and space consumption caused by dead loops in the insertion process and re hashing, the advantages of load balancing of the cuckoo hash algorithm are maximized.

This article conducts experiments by collecting multiple sets of phone number datasets, and finally observes the probability of dead loops occurring under different multiples of the number of data items in the preset hash table space, as well as the spatial performance, insertion performance, query and deletion performance of the hash table, and summarizes and analyzes them.

Keywords: Load balancing, SHA-2, Cuckoo hash algorithm, Cryptography

目录

1	绪论	1
1.1	选题背景和意义	1
1.2	研究现状	1
1.3	论文结构	3
1.4	本章小结	3
2	相关背景知识介绍	4
2.1	SHA-256 加密算法	4
2.2	布谷鸟哈希算法	7
2.3	本章小结	9
3	布谷鸟哈希的应用	10
3.1	电话号码管理系统	10
3.2	数据结构与算法	10
3.3	本章小结	13
4	性能评估	14
4.1	实验环境	14
4.2	参数设置与性能指标	14
4.3	实验设计	15
4.4	结果分析	15
4.5	本章小结	20
5	总结与展望	21
5.1	总结	21
5.2	展望	21
	参考文献	22
	致谢	23

插图目录

1.1	哈希函数简单模型	2
1.2	哈希碰撞简单模型	2
2.1	轮函数流水线模型	6
2.2	传统布谷鸟哈希表简单插入模型	8
3.1	TNMSCH 简单模型	11
4.1	负载因子测试结果	16
4.2	死循环因子测试结果	17
4.3	插入代价测试柱状图	18
4.4	查询代价测试柱状图	19
4.5	删除代价测试柱状图	19

表格目录

2.1 缓冲区寄存器初始赋值表	5
---------------------------	---

1 绪论

1.1 选题背景和意义

随着大数据时代的到来,信息网络飞速发展,人们的日常生活与社会的各行各业产生的数据量呈爆炸式增长。无论是国家公有数据库还是大中小企业的管理系统,服务器计算机需要快速精确地对复杂大规模的流量数据进行存储和分析。过去十年中,全球社交媒体的用户数量达到了 45 亿,其中 Facebook 是所有平台中订阅人数最多的。2021 年 Facebook 的全球每月用户数量约为 29 亿,紧随其后的 Instagram 也达到了 11 亿^[1]。截至 2019 年第一季度,中国腾讯公司开发的社交平台微信的月活跃用户也超过了 11 亿^[2]。社交平台一般通过注册使用的智能手机电话号码作为索引,从而记录用户在社交平台上上传图片,发表动态,收藏视频号和使用小程序等行为特征^[3],实现个人信息的存储的同时方便平台提供即时性服务,并帮助运营公司规划战略展望。但近些年来,以传统的数据结构建立的数据库在处理激增的网络密集型数据时却表现出许多不足^[4]。如何利用有限的时间和空间资源来快速准确地处理激增的用户数据,为数据捕获、数据存储、数据分析和数据可视化^[5]、云计算和密码学等领域^[6]设立了新的挑战。探索并建立能够对数据进行更有效加密存储,更高效查询的数据结构,已成为当下的不时之须。

1.2 研究现状

键值存储系统被广泛用于类似于社交平台的扩展类企业的大规模数据存储^[7],它以键值对的形式来存储数据。当前国内外研究者们针对键值存储问题涌现出了一批优秀的研究成果,以键值对为索引设计的数据结构类型中,数组,链表,二叉查找树和哈希表是最常见的。其中,哈希表能够直接访问内存地址,直接实现对存储数据的查找和删除,是极具潜力的数据结构。

如图1.1所示,通常情况下,哈希表是预先建立的数组,其填充依赖于哈希函数。待插入的数据集包含的每条数据,都具有规定作为“键 (*key*)”的部分,和需要存储的数据部分 (*value*),每条数据的键的内容和长度不一。哈希函数通过把键映射为固定长度的哈希值,从而将键值对 (*key, value*) 插入到哈希表中对应地址的空位 (*bucket*),该键具有索引属性。对于理想化的哈希表来说,它的空间复杂度是 $O(n)$,且对哈希表执行查找,插入和删除操作的时间复杂度都达到了 $O(1)$ 的常数级水准。因此,面对空间资源有限,并时常对数据库进行访问的情况,常数级的时间复杂度让选择哈希表作为存储结构具有较大优势。

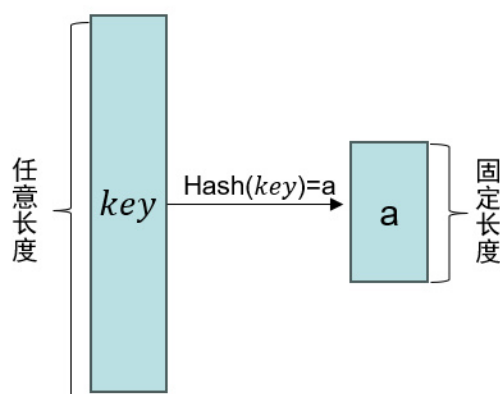


图 1.1 哈希函数简单模型

但在哈希表的插入过程中，会不可避免地遇到哈希冲突。如图1.2所示，对于连续输入的 key_1 和 key_2 ，若通过同一个哈希函数计算出的哈希值相同，都为 b ，那么按输入顺序，哈希表的空位会先被 key_1 占据， key_2 在插入过程中就会发生碰撞。我们把已经填充的空位数与哈希表表长的比值称为负载率，随着负载率的升高，产生上述碰撞的可能性会不断增大，使得哈希表的各项性能出现下降。

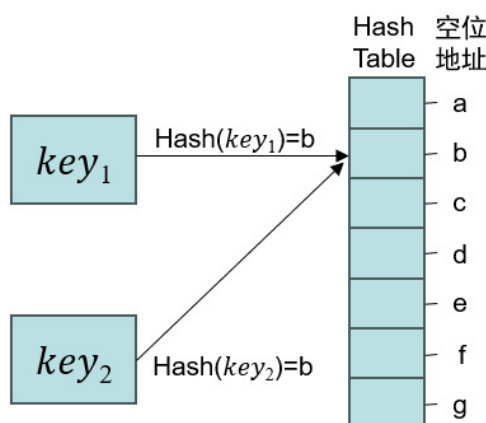


图 1.2 哈希碰撞简单模型

要缓解哈希冲突，一方面则要求使用的哈希函数是强单向性的。即对于不同的键输入，尽可能保证得到的映射结果是哈希值长度范围内的随机值，这需要选择合适的算法作为强性能的哈希函数。

另一方面则需要优秀的寻址算法。在传统的解决方案中，开放定址法将碰撞键值对按照既定次序插入到原地地址前后所能找到的空位，从而提高哈希表的负载率，但容易造成数据的大量堆积现象，且查询复杂度上升；链地址法增加了哈希表的空间复杂度；再哈希法则会令插入步骤消耗额外的内存与时间^[8]。

同时，如果根据数据集的具体规模做出合理预测，从而预先建立大小适当的哈希表数组，能够达到占用内存小，负载率高，查询速度快的目标，具有较强的泛

用性。

1.3 论文结构

为了按条理规整阐述本文所完成的工作，将对论文的总体结构作如下说明：

第一章为绪论部分，主要介绍了哈希函数和哈希表的概念，和基于键值对来构建存储系统过程的研究背景和研究现状。

第二章为相关工作介绍了加密算法 SHA-256 的模型和传统布谷鸟哈希表的数据结构，并对布谷鸟哈希算法的原理进行了分析，对本文的应用工作进行了引申。

第三章提出了布谷鸟哈希算法的应用场景，电话号码管理系统的介绍，详细介绍了该系统的数据结构模型，结合 SHA-256 函数和布谷鸟哈希表，同时增加辅助链表；并分别对该系统实现插入功能，查询功能和删除功能的算法做出阐述。

第四章为针对映射系统的性能评估设计的实验，将整理好的电话号码数据集作为实验输入，通过更改预先的哈希表长度，记录每一次负载率和回收链表的长度，得到合适的哈希表预设长度。

第五章总结了测试实验和调研工作，提出了欠缺和不足之处，并对今后的工作做出展望。

1.4 本章小结

1.4 本章综述本章全面介绍了本章的研究背景，选题意义，以及相关的研究现状。首先，文章指出，在大数据时代来临之时，信息网络发展迅猛，数据量呈现爆炸式增长趋势。

本章对本文的研究背景、选题意义以及研究现状做出了全面介绍。首先，本章指出在大数据时代来临之时，信息网络的拓展与发展迅猛，数据量飞速增长，特别是社交平台等大规模用户数据的存储与处理面临的挑战。随后，对于键值存储系统的普遍应用进行了介绍，强调了哈希表作为一种高效的数据结构，但也存在哈希冲突等问题。接着，讨论了哈希函数的特点，以及包括选择具有强单向性的哈希函数和设计优秀寻址的算法在内的缓解哈希冲突的方法。最后，概述了本文的论文结构，包括相关工作、布谷鸟哈希算法的应用、性能评估实验和总结展望等内容，为后续章节的内容奠定了良好的基础。

2 相关背景知识介绍

本章将重点讲述与本文研究密切相关的背景知识，先介绍 SHA-256 加密函数，再介绍传统布谷鸟哈希算法。首先，本文将详细探讨 SHA-256 算法的原理和流程，该算法是一种安全系数极高的加密函数，被广泛应用于密码学和信息安全领域。其次，本章将逐步剖析一种能有效解决哈希冲突的算法——布谷鸟哈希算法，并明确其通过多哈希桶映射和回溯机制实现常数级的插入、查询和删除复杂度的原理。然而，布谷鸟哈希算法在实际应用中仍存在一些挑战，本章将讨论其可能出现的死循环问题以及对系统性能的影响。通过增加回收链表辅助结构，旨在提高布谷鸟哈希表的插入效率，减少死循环的发生，进一步发挥其在实际应用场景中的优势。

2.1 SHA-256 加密算法

加密函数是一类具有认证性和强不可逆性特质的函数^[9]，SHA 系列函数是常用于数据加密的加密函数，SHA-0 在 1993 年由美国 NIST 设计和提出，被作为联邦信息处理标准首次公布^[10]，但由于安全性能不符合预期，很快就被新修订的 SHA-1 功能所替代。SHA-1 可以将任意长度小于 2^{64} 比特的输入信息作为输入，输出 160 位比特固定长度的密文。为了继续加强加密函数的安全性，拓大加密函数的应用范围，2001 年，美国标准技术研究所对 SHA 系列函数再次进行更新，SHA-2 系列函数^[11]就此诞生，该系列包括 SHA-224，SHA-256，SHA-512 等。之后在 2005 年，较为有效的查找 SHA-1 函数碰撞的方法被山东大学王小云教授发现，使 SHA-1 函数的安全性有所降低^[12]。因此 SHA-256 虽然比 SHA-1 复杂但泛用性更强。

相比于 SHA-1，SHA-256 的映射结果为 256 比特的定长密文，是 SHA-2 系列中被公认具有强安全性和高实用性的加密函数，它在当下的密码学保密领域以及信息安全领域之中得到广泛使用。

SHA-256 的算法流程如下：

步骤 1：补位填充。输入报文 J_0 （报文 J_0 的长度小于 2^{64} 比特）。在 J_0 尾部附加填充字段得到报文 J_1 ，填充字段的第一位数字是 1，其余位均为 0。 J_1 的长度为 L 比特，并且要满足 $L \bmod 512 = 448$ 。

步骤 2：长度附加。用一段 64 比特长的字段记录报文 J_0 的长度，并附加到完成步骤后 1 得到的报文 J_1 的末尾，得到字段 J_2 ，则 J_2 的比特长度必是 512 的整

数倍。

步骤 3: 分组处理。将 J_2 以 512 比特为单位分成 n 组，将每一组长度为 512 比特的数据再分为 16 个 32 位整数字段，按顺序记为 $W_0, W_1, W_2 \dots W_{15}$ ，并继续将其扩展至 W_{63} ，即每组 512 比特的数据对应 64 个字段。对字段 W_i ，当 i 大于 15 时，需要借助旋转移位函数 S_0, S_1 ，按照如下公式进行扩展：

$$W_i = S_1(W_{i-2}) + W_{i-7} + S_0(W_{i-15}) + W_{i-16} \quad (2.1)$$

$$S_0(x) = ROTR^7(x) \oplus ROTR^{18}(x) \oplus SHR^3(x) \quad (2.2)$$

$$S_1(x) = ROTR^{17}(x) \oplus ROTR^{19}(x) \oplus SHR^{10}(x) \quad (2.3)$$

接下来，使用 8 个 32 位的寄存器 (A, B, C, D, E, F, G, H) 作为缓冲区，并按十六进制将其初始化赋值，它们将用于处理分割好的 32 位整数字段的相关计算，和暂时记录计算过程产生的中间值，以及存储最后的输出结果。缓冲区寄存器的初次赋值如表 2.1 所示：

表 2.1 缓冲区寄存器初始赋值表

缓冲区寄存器	十六进制 32 位数初始值
A	0x6a09e667
B	0xbb67ae85
C	0x3c6ef372
D	0xa54ff53a
E	0x510e527f
F	0x9b05688c
G	0x1f83d9ab
H	0x5be0cd19

对已扩展成 64 个 32 位整数的 n 组报文分别进行如图 2.1 所示的轮函数流水线进行 64 轮迭代计算。每一组的 64 轮迭代结果，成为下一组的 64 轮迭代的缓冲区寄存器的初始值，其中， K_i 是相应循环内给定的常量。

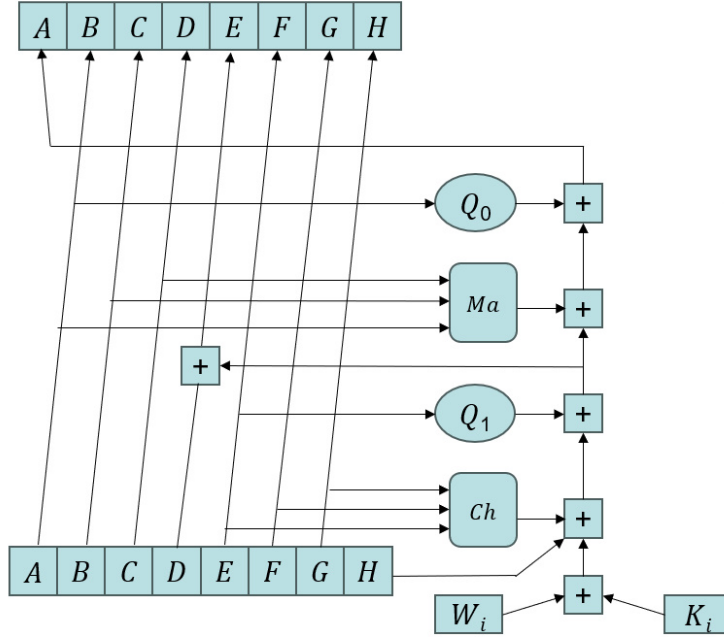


图 2.1 轮函数流水线模型

选择函数 Ch , 多维函数 Ma , 以及 Q_0 , Q_1 的函数逻辑表达式如下

$$Q_0 = ROTR^2(A) \oplus ROTR^{13}(A) \oplus ROTR^{22}(A) \quad (2.4)$$

$$Q_1 = ROTR^6(E) \oplus ROTR^{11}(E) \oplus ROTR^{25}(E) \quad (2.5)$$

$$Ch = (E \vee F) \oplus ((\neg E) \wedge G) \quad (2.6)$$

$$Ma = (A \wedge B) \oplus (A \wedge C) \oplus (B \wedge C) \quad (2.7)$$

步骤 4: 结果输出。分割好的 512 位比特长度的分组字段全部经过迭代计算后，最后输出 256 位比特长度的哈希值就是 SHA-256 加密结果。

综上所述，由于 SHA-256 将输入数据细分成了若干组报文，由将每组依次报文经过了 64 轮错综复杂的运算，使最后得到的哈希值具备很强的不可逆性，并很大程度上保证了映射到 256 位比特空间的随机性。

SHA-256 的安全系数极高，难以破解，自公布多年以来一直成为加密函数界中的常青树。它显然满足了构建哈希表对强性能哈希函数的需要，设计以 SHA-256 为内核的哈希函数是将索引 key 映射为地址的良好选择。

2.2 布谷鸟哈希算法

传统布谷鸟哈希算法的模型是由 Pagh 等人在 2004 年提出的,旨在有效解决哈希冲突^[13]。相较于传统的寻址方式,布谷鸟哈希算法从哈希表的结构上进行了改进,理论上当布谷鸟哈希表的结构足够完美时,可以彻底解决哈希冲突并保证插入查询,删除,操作的复杂度是常数级的。布谷鸟哈希使用 M ($M>1$) 个散列函数,对每个索引键 key 使用 M 个哈希函数进行哈希使得其键值对可以在 M 个哈希桶中各有一个存储空位。通常情况下, M 的值取 2。即通过对输入的每一个键值对的 key 进行两次不同的哈希,对应的,构建两个哈希桶 T_1 和 T_2 。每一键值对 $(key, value)$ 对应 2 个哈希桶,会有 2 个备选位置可以填入。键值对插入哈希表的操作分为以下几个步骤:

步骤 1: 对键值对 $(key, value)$ 的 key 进行两次哈希计算,得到在 T_1 的位置 p_1 , 在 T_2 中的位置 p_2

步骤 2: 如果两个位置皆为空,需要随机选择一个位置并将键值对插入。

步骤 3: 若其中一个位置被其他键值对占据,则插入未被占据的另一个位置。

步骤 4: 若 key 在 T_1 的位置 p_1 和在 T_2 中的位置 p_2 都被其他键值对占据,则随机踢出其中一个。对被踢出的新键值对,再根据其 key 计算其在另外一个哈希桶的备选位置,当这个备选位置为空时,插入该新键值对,若不为空,将再次踢出先前占据着该备选位置的旧键值对并插入新键值对,将被踢出的旧键值对,将它做为新键值对并根据其 key 寻找它在另一个哈希桶的备选位置,如此循环,直到有一个踢出来的键值对在插入它的另一个备选位置时没有再发生新旧键值对的碰撞为止。

步骤 5: 若步骤 4 的循环次数超越既定的阈值,则判断这一轮键值对的插入出现死循环。立刻扩容哈希表得到新哈希表,之前还没有完成全部键值对插入的旧哈希表被销毁,将键值对重新哈希并插入新哈希表。

作为一种创新的哈希冲突解决方案,布谷鸟哈希算法有很多可取之处,也有很大的应用价值潜力。首先,相对于传统的哈希表结构,通过使用多个哈希函数,布谷鸟哈希算法可以为哈希表中的各个键值对提供多个备选位置,从而有效地令哈希冲突的更难发生。这种设计使得插入、查询和删除等操作的时间复杂度可以保持在常数级别,从而提高了系统的性能和效率。另外,布谷鸟哈希算法的灵活性也为其在不同场景下的应用提供了可能。通过调整哈希桶的大小和哈希函数的数量,可以根据具体的需求和数据特征来优化系统性能。例如,在处理电话号码等关键信息时,可以根据号码的特点和数据规模,灵活设置哈希函数的数量和参数,

以达到最佳的存储和查询效果。

此外，布谷鸟哈希算法的机制给予其扩展和容错的空间。当系统中的数据规模增大或者数据分布发生变化时，可以通过扩容和重新哈希的方式来调整哈希表的大小和结构，从而保持系统的稳定性和性能。布谷鸟哈希算法作为一种新颖的哈希冲突解决方案，具有广阔的应用前景和研究价值。未来可以进一步探索其在不同领域的应用，如数据库管理、网络路由和信息安全等，以及与其他数据结构和算法的结合，从而推动其在实际应用中的发展和应用。

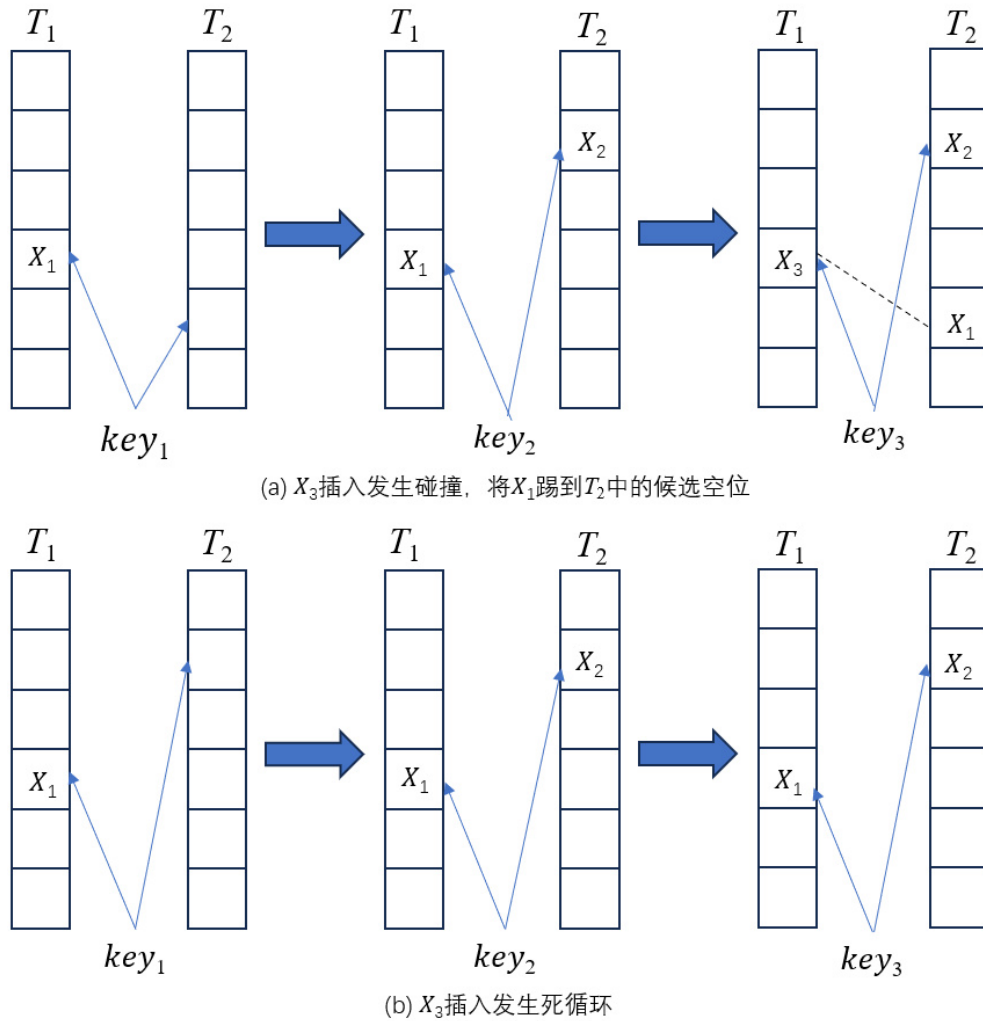


图 2.2 传统布谷鸟哈希表简单插入模型

图2.2模拟了布谷鸟哈希表执行一项键值对插入时面对的各种分支情形，假设键值对 X_i 的索引键为 key_i 。可以看出，在 (a) 中，对 key_1 进行两次哈希，随机选择一个空位， X_1 插入到哈希桶 T_1 中。插入 X_2 时， key_2 在 T_1 的空位被占，于是选择填入 T_2 。插入 X_3 时，两个空位都被占据，于是随机踢出了 T_1 中的 X_1 ， X_1 则填入其在 T_2 中的候选空位将。而在 (b) 中，连续输入的 key_1 ， key_2 ， key_3 在 T_1 ， T_2 中的映射位置恰好相同，在 key_3 的插入过程中，就会出现不停踢出键值对

的死循环现象。

如果对布谷鸟哈希表的执行查询操作，只需要对 *key* 进行两次哈希得到两个哈希值地址，查看两个地址中是否存储有键值对，之后判断所存储的键值对是不是具有相同的 *key*。如果不存在或不相同，就返回查询失败，如果相同，则返回查找成功，并输出 *value*。

如果对布谷鸟哈希表执行删除操作，只需要在查询成功的基础上，再清空相应的存储空位。显然，查询和删除的复杂度是相同的。

综上可得，完美的布谷鸟哈希表具有常数级的插入，查询和删除复杂度，并能花费较少计算利用较多内存。但实际情况中，随着哈希表负载率和遇到死循环的概率是成正相关的。而一旦执行哈希表扩容和重新哈希操作，之前的内存开销和插入流程将全部作废，造成时间和空间的浪费。

本文的主要工作是改进布谷鸟哈希算法的流程，增加一个线性链表作为辅助结构，将其应用于以电话号码为索引的用户信息存储系统中。在消除插入过程出现死循环导致再哈希造成的时间空间消耗条件下，提高布谷鸟哈希表的插入效率，并更大程度上发挥布谷鸟哈希算法负载均衡的优点。

2.3 本章小结

本章主要介绍了两个与本文研究密切相关的背景知识，分别是 SHA-256 加密算法和布谷鸟哈希算法。首先，对于 SHA-256 加密函数，本文按步骤介绍了其算法流程，实现原理。其次，传统布谷鸟哈希算法这一种解决哈希冲突的有效方法，也在本文中进行了详细解构。该算法通过将键值对映射到多个哈希桶，并使用回溯和重新哈希等技术来解决插入过程中可能出现的死循环问题，从而实现了常数级的插入、查询和删除复杂度。然而，实际应用中，布谷鸟哈希算法仍然存在一些不足之处，如插入过程中可能出现的死循环问题以及由此带来的再哈希操作的时间和空间消耗。

在此基础上，本文将提出一种改进布谷鸟哈希算法的方案，并将其应用于电话号码为索引的用户信息存储系统中。通过增加回收链表辅助结构，本文旨在提高布谷鸟哈希表的插入效率，减少死循环的发生，进一步发挥布谷鸟哈希算法的优势，从而更确切地满足不同实际应用场景中对于数据存储和查询效率的需求。综上所述，本章对于本文后续工作的理论基础和技术支持起到了重要作用，为后续章节的具体研究内容提供了必要的理论和方法指导。

3 布谷鸟哈希的应用

本章将探讨布谷鸟哈希算法在电话号码管理系统中的应用。在现实生活中,许多中小型企业和管理机构都使用电话号码作为主键建立数据库,用于存储员工记录 and 用户信息。然而,传统的数据存储结构存在着插入、查询和删除等操作复杂度较高的问题。为了解决这些问题,本章将引入布谷鸟哈希算法,并以此为基础构建电话号码管理系统。本章将详细介绍该系统的数据结构与算法,以及如何利用布谷鸟哈希算法实现高效的电话号码管理。

3.1 电话号码管理系统

在现实生活中,大量的中小型企业和管理机构通过员工和用户手机号码作为索引主键建立数据库,对员工记录 and 用户信息进行存储,并在后续时常会对数据库进行插入,查找和删除操作。传统的数据存储结构都存在一项或多项操作的复杂度较大的问题:对于数组和链表而言,本章必须以线性的方式进行查找,即使令键值对在数据结构中有序排列,使用二分查找的复杂度也是对数级的,倘若投入到在实际应用代价太大。此外,绝大多数中国手机通话运营商下的电话号码由 11 位数字组成(极少数是 10 位),如果为了让插入,查找,删除的复杂度为 $O(1)$ 而直接将电话号码作为索引地址去开辟相匹配数量级空间的数组,是更不可取的,且电话号码的连续性非常差,两个有效电话号码之间存在大量空位几乎是必然,这就造成了极大的空间浪费。因此使用哈希表的数据结构来存储电话号码和用户信息的键值对具有优势。

本章的目的是构建一个布谷鸟哈希电话号码管理系统 (Telephone Number Managing System by Cuckoo Hashing) 后简称为 TNMSCH。将待处理数据集中的 11 位数电话号码作为主键 *key*, 通过强性能哈希函数将其尽可能稠密地映射在一个较小空间的哈希表中,从而实现负载均衡和达到高空间利用率,且要保证该数据结构的三项性能都足够强大。这与 SHA-256 散列函数以及布谷鸟哈希算法的性质相契合,本章将以此为基础构建符合预期的数据结构。

3.2 数据结构与算法

TNMSCH 由如图 3.1 所示的哈希结构与链表结构组成,链表结构用于存储发生死循环的键值对。

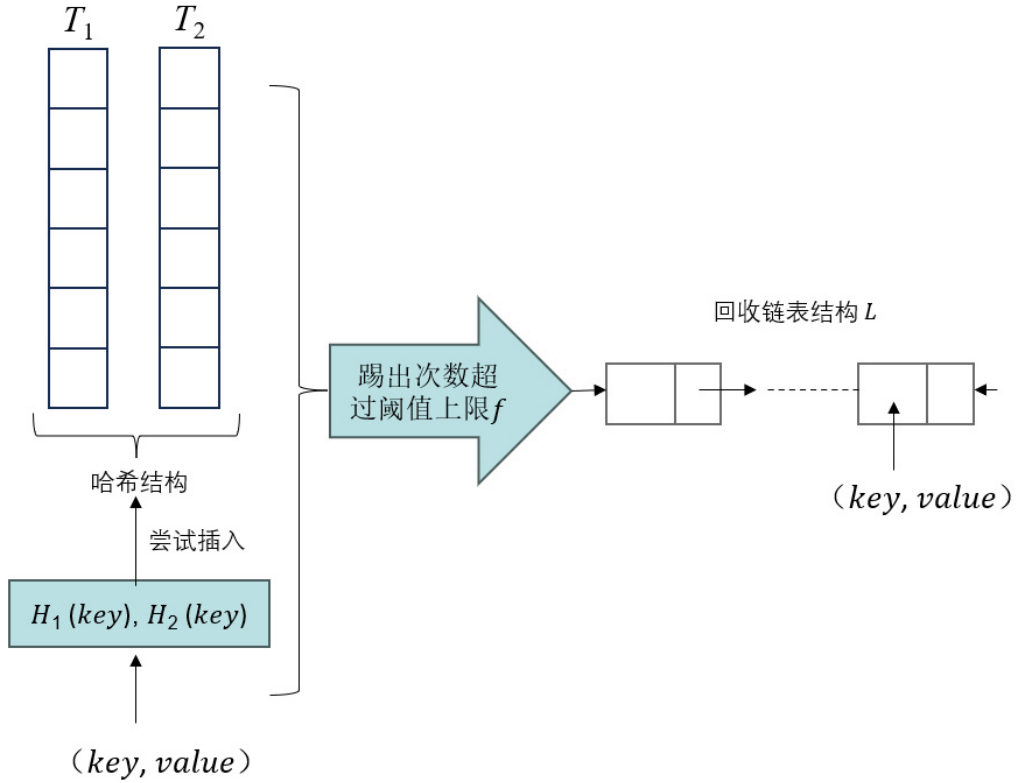


图 3.1 TNMSCH 简单模型

对于哈希结构，使用布谷鸟哈希算法构建哈希桶 T_1 , T_2 。基于 SHA-256 函数的强性能和映射序列的强随机性，本章设计对应的哈希函数 H_1 和 H_2 。考虑到 11 位电话号码中尚未出现以 2 开头的号码，以及显示预期的中小型企业人数的基数大小。设计哈希函数 $H_1(key)$ 取 SHA-256(key) 的 16 进制哈希值的前 32 位 mod 哈希桶 T_1 预设长度 l_1 ，哈希函数 $H_2(key)$ 取 SHA-256(key) 的 16 进制哈希值的后 32 位 mod 哈希桶 T_2 预设长度 l_2 。

对于链表结构，使用一个额外的单链表 L 作为辅助结构，它用于回收存储在插入哈希表过程中踢出次数超过阈值 f 而被判定为死循环的键值对。由于回收链表 L 的存在，TNMSCH 不再实施传统的哈希表扩容并重新哈希的操作流程，插入消耗被大幅削减了。但 L 的最终长度 l_3 对数据结构的查找和删除性能也会产生小幅影响。

3.2.1 插入操作

对该电话号码管理系统的插入算法按顺序以如下步骤进行：

- (1) 开始一个键值对 $(key, value)$ 的插入，初始化计数器 $count = 0$,
- (2) 若 $count$ 大于阈值上限 f ，直接进入第 (6) 步，
- (3) 将键值对中的 key 值经过函数 H_1 求得相应的哈希值 $p_1 = H_1(key)$ ，判断

T_1 中 p_1 是否为空位, 若为空, 将键值对插入, 直接进入下一轮键值对的插入。否则进行第 (4) 步,

(4) 将键值对中的 key 值经过函数 H_2 求得相应的哈希值 $p_2 = H_2(key)$, 判断 T_2 中 p_2 是否为空位, 若为空, 将键值对插入, 直接进入下一轮键值对的插入。否则进行第 (5) 步,

(5) 若 $count \bmod 2 == 0$, 则根据已经求得的哈希值 $p_1 = H_1(key)$ 进行插入, 若 T_1 的 p_1 位置为空, 将键值对填入, 进行下一轮插入, 若不为空, 则踢出原有键值对, 继续插入键值对, 且 $count = count + 1$, 踢出的键值对作为 $(key, value)$ 进入第 (2) 步; 若 $count \bmod 2 == 1$, 则根据已经求得的哈希值 $p_2 = H_2(key)$ 进行插入, 若 T_2 的 p_2 位置为空, 将键值对填入, 进行下一轮插入, 若不为空, 则踢出原有键值对, 继续插入键值对, 且 $count = count + 1$, 踢出的键值对作为 $(key, value)$ 进入第 (2) 步,

(6) 以按表头插入的方式, 将键值对插入回收链表 L 。进入下一轮插入。

3.2.2 查询操作

对给定的键 key , 查询相应的键值对 $(key, value)$, 在该电话号码管理系统的操作如下:

(1) 将 key 经过哈希函数 H_1 , 得到哈希值 $p_1 = H_1(key)$, 判断 key 是否与 T_1 中 p_1 位置的键值对的 key 相等, 相等则输出 $value$, 查找结束, 否则进入第 (2) 步,

(2) 将 key 经过哈希函数 H_2 , 得到哈希值 $p_2 = H_2(key)$, 判断 key 是否与 T_2 中 p_2 位置的键值对的 key 相等, 相等则输出 $value$, 查找结束, 否则说明要查询的键值对不存在于哈希表结构中, 进入第 (3) 步,

(3) 在线性链表 L 中正向查找索引 key , 若查询到, 则输出对应的 $value$ 。若查询不到, 返回查询失败。

3.2.3 删除操作

进行键值对的删除操作时, 首先要对 TNMSCH 的哈希结构的 T_1, T_2 进行查询操作, 若 key 吻合, 则进行桶内部清空操作; 若 key 不吻合或哈希表中对应的位置是空位, 则对回收链表执行正向查找, 找到前驱节点并删除; 若哈希表和链表中都未找到吻合的 key , 返回删除失败。

3.3 本章小结

本章介绍了布谷鸟哈希算法在电话号码管理系统中的应用。首先，本章讨论了在现实生活中使用电话号码作为主键建立数据库的需求背景，并指出传统数据存储结构存在的问题。随后，本章详细介绍了电话号码管理系统的数据结构与算法，包括哈希结构和链表结构的设计，以及插入、查询和删除操作的具体实现方法。通过引入布谷鸟哈希算法，本章实现了高效的电话号码管理系统，提高了数据操作的效率，并减少了系统的空间浪费。本章内容为后续章节的实验工作奠定了基础，为 TNMSCH 的实际应用提供了理论支持。

4 性能评估

本章将对布谷鸟哈希算法在 TNMSCH 中的性能进行全面评估。本章将介绍实验环境的搭建，包括硬件配置和数据集的采集情况。随后，将详细阐述参数设置和性能指标的定义，以及实验设计的方案。最后，本章将进行实验结果的分析，包括空间性能、插入性能以及查询和删除性能，以便全面评估布谷鸟哈希算法在 TNMSCH 中的适用性和效率。

4.1 实验环境

本次测试的硬件部分使用搭载了 24 核 (32 线程, Intel Core i9@5.80GHz) 的核心处理器, 128Gb 内存的 Windows 系统的电脑作为实验机器, TNMSCH 的数据结构与算法使用 python 语言实现。

本章从 GITHUB 平台采集到了 30 多万项含有电话号码的脱敏用户数据, 每项数据记录有用户的移动电话号码, 开号地址, 号码运营商以及邮政编码。并随机抽取其中的 100000 项, 将电话号码作为主键 *key*, 剩余部分作为 *value*, 构建键值对 (*key, value*) 作为测试集 *dataset1*, 从 *dataset1* 中再以随机抽样的方式抽取 20000 项作为测试集 *dataset2*。

接下来, 通过改变预设相关参数, 进行多轮实验, 每轮先使用 *dataset1* 进行插入测试, 然后使用 *dataset2* 依次进行查询和删除测试, 并记录实验结果。

4.2 参数设置与性能指标

通过预先设置和更改以下参数, 从而完成平行实验:

设置 α 为构建哈希结构时两个哈希桶的空位数之和与需要插入的测试集的键值对总数的比值, 它可以用来表示哈希结构预支空间的大小。

设置 f 为碰撞次数既定阈值, 当一个键值对的插入过程产生了超出 f 次的踢出, 判定为死循环。

通过定义以下因素为性能指标, 是实验的输出部分, 用以衡量 TNMSCH 的各项性能:

L_f (Load factor) 是哈希桶的负载因子, 为已插入哈希桶的键值对数与构建哈希结构时两个哈希桶的空位数的比值, 用于评估空间性能。

D_f (Death factor) 是死循环因子, 记录一轮插入后产生的回收链表的长度, 用于评估空间性能。

Icr (Insert cost rate) 是插入代价, 是每轮实验插入过程中访问内存的总次数与插入 key 数的比值, 即平均插入访问次数, 用于评估插入性能。

Fcr (Find cost rate) 查询代价, 是每轮实验查询过程中发生的对内存的总访问次数与测试集 key 数的比值, 即平均查询访问次数, 用于评估查询性能

Dcr (Delete cost rate) 删除代价, 是每轮实验删除过程中发生的对内存的总访问次数与测试集 key 数的比值, 即平均删除访问次数, 用于评估删除性能。删除操作对内存的访问方式与查询操作相似, 将放在一起进行结果分析。

4.3 实验设计

进行 12 轮实验: 设置 α 为自变量, 0.2 为单位间隔, α 在 [1.0, 2.0] 的范围内取值, 进行 6 轮实验, 以 0.5 为间隔, α 在 [2.5, 5.0] 的范围内取值, 进行 6 轮实验。每轮实验分为两步, 且每轮的死循环判定阈值都设置为 $f = 500$ 。

第一步以 dataset1 中的键值对作为输入, 进行插入操作, 输出性能指标 L_f , D_f , Icr , 第二步以 dataset2 中的 key 作为输入, 先进行查询操作, 输出性能指标 Fcr , 再进行删除操作, 输出性能指标 Dcr 。

记录总共 12 轮实验每一轮产生的五项性能指标, 将每个指标的每轮输出与该轮 α 进行对应, 对每个性能指标以 α 为自变量进行 12 轮实验输出的对照分析。

4.4 结果分析

本章将全面评估所提出的布谷鸟哈希算法在空间性能、插入性能、查询和删除性能三个方面的表现, 以验证其在解决哈希冲突和优化数据存储管理方面的有效性。

首先, 将对 TNMSCH 的空间性能进行评估。通过分析哈希表的负载因子和死循环因子, 可以了解到在不同的哈希表大小下, 算法的空间利用率和冲突处理效率。通过观察负载因子和死循环因子随着哈希表大小的变化趋势, 可以确定最佳的哈希表大小设置, 最大程度地减少死循环发生率, 同时将存储空间的利用率保持在较高水准。

其次, 将对 TNMSCH 的插入性能进行评估。通过测量插入操作的平均踢出次数和插入代价率, 可以评估算法在不同负载下的插入效率和性能。插入性能的评估结果将直接影响到系统的实时数据更新和存储操作, 因此对于保证系统稳定性和实用性至关重要。

接着, 将对 TNMSCH 的查询和删除性能进行评估。通过测量查询和删除操作的平均内存访问次数和操作代价率, 可以评估算法在不同负载下的查询和删除效

率和性能。查询和删除性能的评估结果将直接影响到系统的数据检索和清理操作，是保证系统运作的高效性和后续的可维护性的依据。

通过对布谷鸟哈希算法在空间性能、插入性能、查询和删除性能三个方面的全面评估，可以全面了解其在实际应用中的表现和优劣，为进一步优化和改进算法提供参考和指导。

4.4.1 空间性能

在 α 的 12 个不同取值下，负载因子的变化情况如图4.1所示，纵坐标为负载因子 L_f ，横坐标为 α 。死循环因子 D_f 的变化情况如图4.2所示，纵坐标为死循环因子 L_f ，横坐标为 α 。

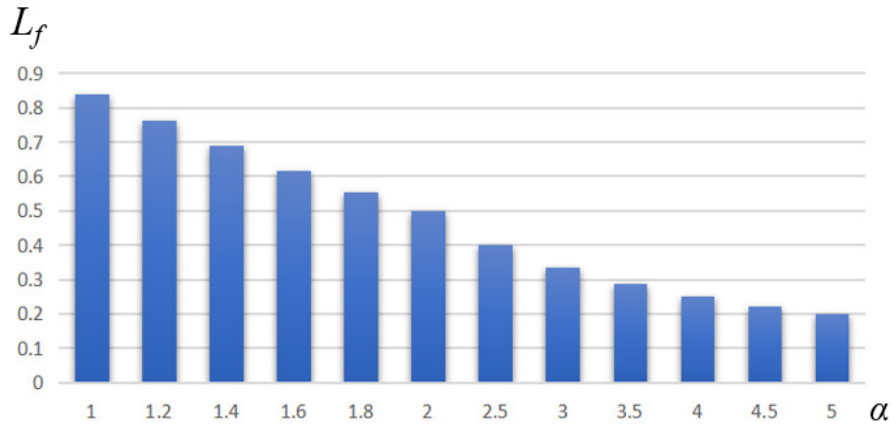


图 4.1 负载因子测试结果

当 α 取 1 时，负载因子最高，取值为 0.8385，死循环因子也最高，达到了 16152，这是因为此时预设哈希表的总空位数与待插入 *key* 数是相等的，预设空间过小，使得插入结束后哈希表中的键值对分布非常稠密，空间的利用率很高，但过小的预设空间也使得当哈希表的负载持续上升时，插入键值对发生哈希冲突的概率增加，导致有 16.152% 的键值对在插入过程中被判定为死循环，辅助链表的长度较大。

当 α 小于 2 时，负载因子和 α 两者呈现出接近反比的关系，这是因为每轮插入中，总有一部分被判定为死循环的键值对不能插入哈希结构，导致负载因子的值总要比 $1/\alpha$ 要小一些。

负载因子和死循环因子也随着 α 的增大而减小，因为随着预设哈希空位的倍数级增加，*key* 通过哈希函数得到的哈希值接近于在一个更大空间里取随机值，键值对在插入过程中发生碰撞的可能降低，从而使得更多键值对被插入哈希结构而非辅助链表。

当 α 增加到 2 时，负载因子等于 0.5，两者从接近反比关系变为完全反比关

系，并随着 α 的增大继续保持完全反比，这是因为此时全部键值对都插入了哈希表，死循环因子也降到了 0。

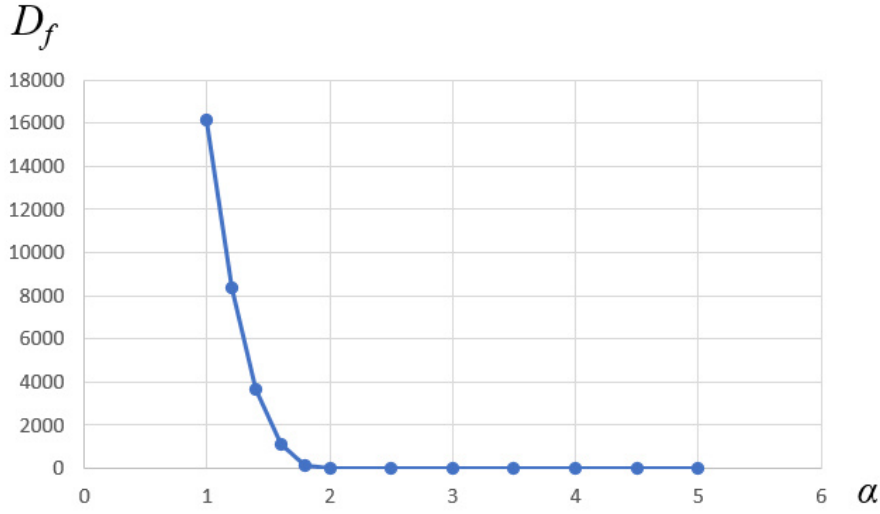


图 4.2 死循环因子测试结果

通过实验可以观察到，在前 5 轮实验中，也就是 α 从 1 增加到 1.8 时，负载因子从 0.8385 下降到了 0.5548 的中等水平，死循环因子却从 16152 下降到了 144，总 key 数为 100000，死循环占总 key 数的比率从 16.152% 下降到了 0.144%，死循环因子发生了大幅度下降，而在后 7 轮实验中， α 从 2 增加到 5 时，负载因子从 0.5 下降到了 0.2 的较低水平，而死循环因子只从 144 下降到了 0，死循环占总 key 数的比率从 0.144% 下降到了 0%，且在 α 为 2 时就已经归零，下降幅度非常小。造成这一现象是因为在前 5 轮实验中，预设哈希空间是从一个空位十分紧缺的状态扩容了一倍，能够缓解大量的哈希冲突，使数量明显的键值对得以成功插入到哈希表结构中，是以负载率下降带来的较小空间代价，换取了键值对在哈希结构中的存储数量的较大提升；但在后 7 轮的实验中，当 α 为 3 时，预设哈希表在插入结束后已经不再是一个较饱和的状态，甚至还有大量未插入键值对的空位，通过持续增大 α 来哈希冲突缓解换来键值对在哈希结构中的存储数的提升已经没有效益，还会带来倍数增长的空间开销。

综上，可以得出，在 α 取 1.8 至 3 时，负载因子均达到了 0.3 以上，而死循环因子的占比均低于了 0.15%，控制在了一个很低的水准。此时，TNMSCH 的有效存储率高，空间利用率较好，空间性能较强。

4.4.2 插入性能

在 α 的 12 个不同取值下，插入代价的变化情况如图 4.3 所示，纵坐标为 dataset1 每个键值对的平均插入内存访问次数 Icr ，横坐标是 α 。

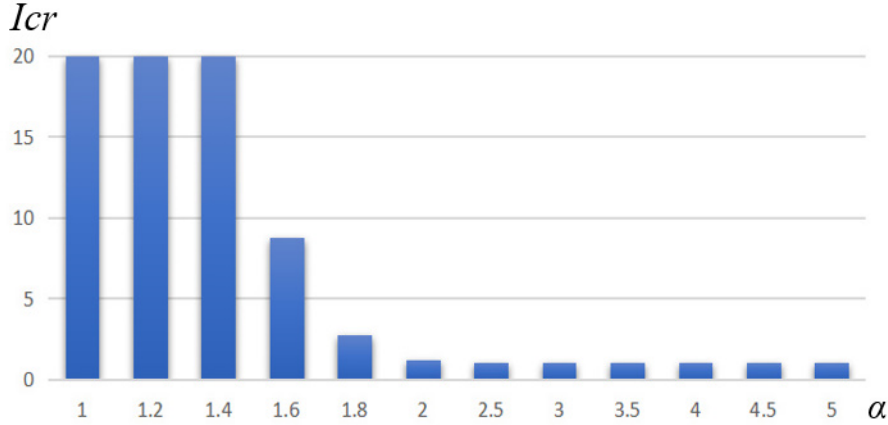


图 4.3 插入代价测试柱状图

可以看出，设置 α 的取值逐渐增大时，平均插入内存访问次数逐渐减少。这是由于随着预设哈希空间的不断增大，插入过程中哈希表中的键值对密度被稀释了，一方面键值对在插入哈希表时候选空位被占发生碰撞的可能性出现下降，一方面使得插入过程中发生死循环键值对减少，迭代踢出键值对的操作次数大幅减少，从而减少了内存访问次数。在前 3 轮实验中，平均插入内存访问次数的取值都超过了纵坐标的上限。

当 α 取值为 1, 1.2, 1.4 时，平均插入内存访问次数的取值均远超过上限 20，达到了 86.0957, 47.3182, 23.0745。这是因为当 α 较小时，插入过程中发生死循环的键值对占比很高，每一个死循环键值对的插入过程对内存的访问次数会超过了设定的死循环判定阈值 $f = 500$ ，也无法忽略其对整体测试集的键值对插入速度带来的影响。当 α 取值为 1 至 1.6 时，随着 α 的增加，哈希冲突缓解效率的显著提升以及死循环键值对数量的大幅减少，使得平均插入内存访问次数连续骤减。 α 继续增加，平均插入内存访问次数减少速率逐渐平缓，直到当 α 取值为 1.8 和 2 时，平均访问次数为 2.7034 和 1.1832，已经达到了接近常数级的时间复杂度。这是因为随着哈希表的容量扩大，哈希冲突产生的期望随之降低，死循环键值对的占比降至很低，死循环键值对减少带来的插入内存访问次数的减少对总体的影响已经很小。当 α 取值为 3.5, 4, 4.5, 5 时，插入代价取值分别为 1.0134, 1.0092, 1.0065 插入代价的下降不明显，一直维持在较低的常数级水准，但 α 的增加会使得空间开销增大。

综上，可以得出，在 α 大于 1.8 时，TNMSCH 的插入时间消耗已经降为常数级，插入性能已经达到了特别优秀的水准。

4.4.3 查询和删除性能

在 α 的 12 个不同取值下, 查询代价的变化情况如图 4.4 所示, 纵坐标是 dataset2 的每个 *key* 的平均查询内存访问次数 Fcr , 横坐标是 α 。删除代价的变化情况如图 4.5 所示, 纵坐标是 dataset2 的每个 *key* 的平均删除内存访问次数 Dcr , 横坐标是 α 。

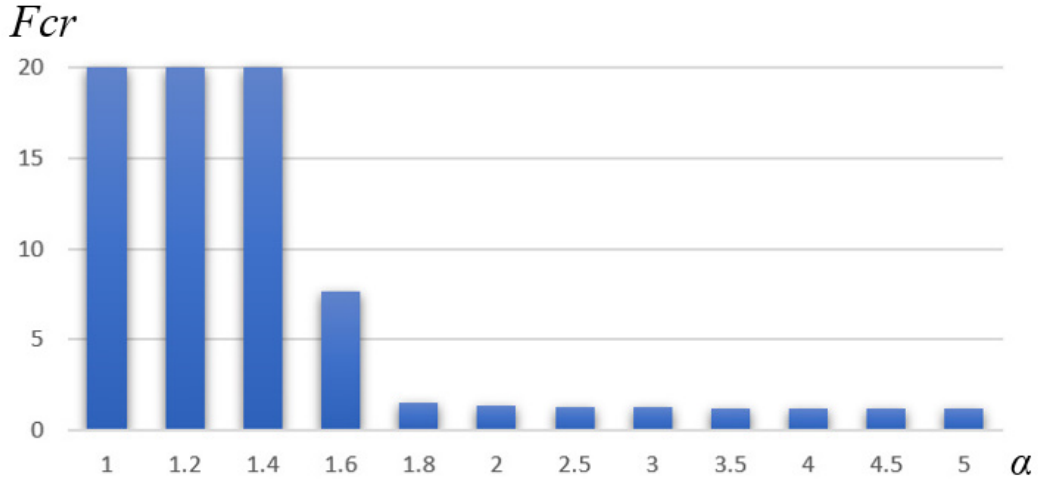


图 4.4 查询代价测试柱状图

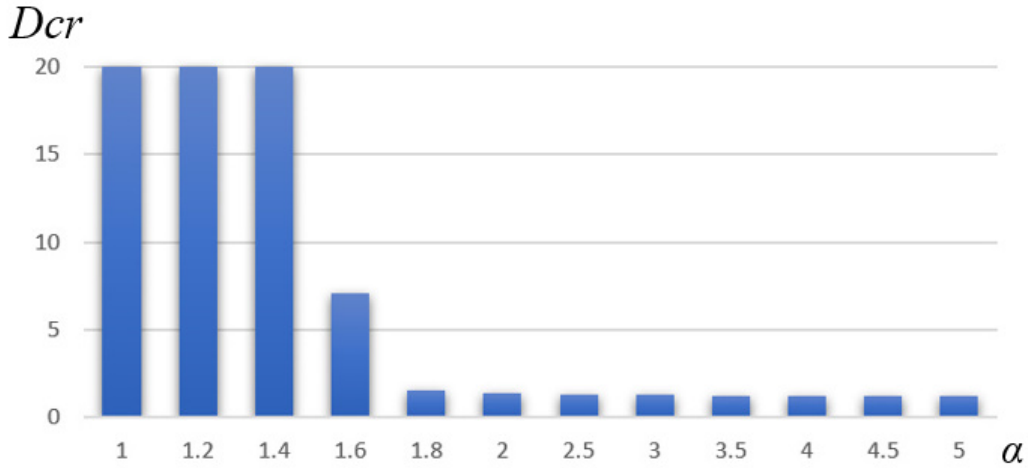


图 4.5 删除代价测试柱状图

可以看出, 随着 α 的增大, 平均查询内存访问次数与平均删除内存访问次数的取值都在逐渐减小, 并且, 它们的取值十分接近。这是由于使用了从插入数据集 dataset1 中随机抽取的项构成的 dataset2, 在 α 小于 1.8 时, 产生的辅助链表长度很长, 有不小比例的键值对的 *key* 在哈希结构中无法查询到它们, 需要到链表中以复杂度为 $O(n)$ 的方式正向查找, 这一过程中对内存的访问次数包括了遍历链表的节点次数, 从而对整体数据集 *key* 的平均查询内存访问次数带来不小的影响, 使

得查询速度慢。在前 3 轮实验中, 平均查询内存访问次数的取值分别为 1297.1949, 353.8229, 76.2653, 以及平均删除内存访问次数的取值分别为 1190.9917, 322.7580, 69.1683, 它们都超过了纵坐标的上限。值得注意的是, 删除操作过程中, 如果访问了链表, 在遍历节点后, 将对用的节点删除, 这会使得链表长度缩减, 使得之后要遍历链表时的内存访问次数产生极其微小的影响, 所以在 12 轮实验中的前 5 轮, 平均查询内存访问次数都要比平均删除内存访问次数高一点点, 它们取值十分接近。

当 α 从 1 增加到 1.8 时, 随着预设哈希表空位的增多, 产生的辅助链表的长度显著缩短, 查询或是删除 *key* 时需要访问链表的可能性很大程度上降低了, 平均查询内存访问次数降低到了 1.5398, 平均删除内存访问次数也降低到了 1.5211, 已经接近了常数级。当 $\alpha=3$ 时, $Fcr=1.2687$, $Dcr=1.2687$, 均低于 1.3, 由于哈希空间已经十分富余, 这使得键值对都存储在了哈希结构中, 查询与删除的速度非常快。

因此, 可以看出, 当 α 大于 1.8 时, TNMSCH 查询代价还有删除代价随着 α 的增大已经没有了明显的变化。总结来看, 此时 TNMSCH 的查询和删除非常快, 其代价接近于理论中完美的哈希表结构。

4.5 本章小结

本章对布谷鸟哈希算法在电话号码管理系统中的性能进行了全面评估。首先, 本章建立了实验环境, 搭建了合适的硬件配置, 并采集了实验所需的数据集。然后, 本章根据实验需求设置了参数并定义了性能指标, 设计了详细的实验方案。接着, 本章进行了两组实验, 分别对不同的哈希表大小进行了插入、查询和删除测试, 并分析了实验结果。最终, 本章得出了布谷鸟哈希算法在电话号码管理系统中的空间利用率高、插入、查询和删除效率优秀的结论, 且在预设哈希表空位数为存储数据集键值对数的 1.8 至 3 倍时, 泛用性较好, 为其在实际应用中的推广提供了有力支持。

5 总结与展望

5.1 总结

本文通过为布谷鸟哈希表增加回收死循环键值对的额外辅助链表，消除了传统布谷鸟哈希在插入过程中执行扩容与重新哈希步骤带来的计算时间和内存空间的消耗。同时通过平行对照实验，观察到了针对数据规模，预先建立哈希表的合适大小。但是哈希表结合链表的存储结构在使得数据结构的维护性和可读性有所降低。

当数据规模较小时，比如针对电话号码的存储，较低的死循环率系数让链表部分的查找复杂度难以影响总体的查找效率。但当数据规模十分庞大，单链表结构对总体查找复杂度带来的影响不容忽视时，本文的数据结构与纯粹的哈希表存储结构在查找性能上会出现较大差距，也是本文研究工作的缺陷，不能应用于存储庞大数量级项数据后的频繁查找。

5.2 展望

尽管本文对布谷鸟哈希算法在电话号码管理系统中的性能进行了全面评估，但仍有一些方面可以进一步探索和改进。首先，可以进一步优化哈希函数的设计，在哈希表的负载率上做改进，以减少哈希冲突对系统性能的影响，从而在整体上提高系统的性能。其次，为更全面地测试系统的性能表现，可以考虑为不同规模和参数设置的系统引入更多的试验数据集。另外，对于其他哈希算法或数据结构的应用在电话号码管理系统上也要进行探索，并与布谷鸟哈希算法进行平行对照。

随着通信技术的日益发展以及用户对系统的要求日益提高，电话号码管理系统将不断面临新的挑战与需求。因此，要不断地开展研究与改进工作，以从整体上满足用户的需求并提高系统的性能表现，以应对日益变化的市场竞争。综合以上情况，今后的研究可着重从算法改进，数据集扩充，多种数据结构比较分析等几个方面着手，使电话号码管理系统的性能和功能有进一步的提高。

总体来看，布谷鸟哈希算法，是高效的哈希算法，具备不容小觑的潜力。随着计算机技术的不断进步和发展，对哈希算法的性能和效率要求也越来越高。布谷鸟哈希算法以其高速的计算效率和较低的资源消耗在性能方面具备一定优势，未来在优化算法结构和提升计算速度方面也有望实现更大突破，相信以布谷鸟哈希算法构建的构建的数据存储结构将会迎来更加美好的发展前景。

参考文献

- [1] SMITH T. An exploratory analysis of the relationship of problematic facebook use with loneliness and self-esteem: the mediating roles of extraversion and self-presentation[J]. *Current Psychology*, 2023, 42(28): 24410-24424.
- [2] CAI Z, LUO X, XU X, et al. Effect of wechat-based intervention on food safety knowledge, attitudes and practices among university students in chongqing, china: a quasi-experimental study [J]. *Journal of Health, Population and Nutrition*, 2023, 42(1): 28.
- [3] LI W, HAN L Q, GUO Y J, et al. Using wechat official accounts to improve malaria health literacy among chinese expatriates in niger: an intervention study[J]. *Malaria journal*, 2016, 15: 1-13.
- [4] 李玮, 张大方, 谢鲲, 等. 一种面向闪存键值存储的矩阵索引布鲁姆过滤器[J]. *计算机研究与发展*, 2015, 52(5): 1210-1222.
- [5] CHEN C P, ZHANG C Y. Data-intensive applications, challenges, techniques and technologies: A survey on big data[J]. *Information sciences*, 2014, 275: 314-347.
- [6] LI Y, GAI K, QIU L, et al. Intelligent cryptography approach for secure distributed big data storage in cloud computing[J]. *Information Sciences*, 2017, 387: 103-115.
- [7] ATIKOGLU B, XU Y, FRACHTENBERG E, et al. Workload analysis of a large-scale key-value store[C]//*Proceedings of the 12th ACM SIGMETRICS/PERFORMANCE joint international conference on Measurement and Modeling of Computer Systems*. 2012: 53-64.
- [8] 冯克清, 曾绍良. HASH 表查找效率的讨论和验证[J]. *工程科学学报*, 2021, 6(1): 183-189.
- [9] DAMGÅRD I B. A design principle for hash functions[C]//*Conference on the Theory and Application of Cryptology*. Springer, 1989: 416-427.
- [10] BIHAM E, CHEN R, JOUX A. Cryptanalysis of sha-0 and reduced sha-1[J]. *Journal of Cryptology*, 2015, 28: 110-160.
- [11] RACHMAWATI D, TARIGAN J, GINTING A. A comparative study of message digest 5 (md5) and sha256 algorithm[C]//*Journal of Physics: Conference Series: Vol. 978*. IOP Publishing, 2018: 012116.
- [12] WANG X, YIN Y L, YU H. Finding collisions in the full sha-1[C]//*Advances in Cryptology—CRYPTO 2005: 25th Annual International Cryptology Conference, Santa Barbara, California, USA, August 14-18, 2005. Proceedings 25*. Springer, 2005: 17-36.
- [13] PAGH R, RODLER F F. Cuckoo hashing[J]. *Journal of Algorithms*, 2004, 51(2): 122-144.

致谢

时光飞逝，日月如梭，历经青春时期三载的拼搏，踌躇满志的我迈进了中山大学的校门，开启了自己的本科生涯。如今，这一段充满欢畅与酸涩，忙碌与坎坷的大学时光，也即将告一段落。

在计算机学院的学习日程里，我第一次接触到了相关学科理论的知识海洋。从头脑懵懂初入程序设计，学习 C++，python 等语言代码的编写，到孜孜不息海量阅读，明确毕业课题，设计实验完成论文，是化解疑惑的蜕变步骤，是难能可贵的成长履历，也是培养科学素养的缓慢过程。随着古今中外的算法宝库进入我个人的视野，身为后辈体会到的更是吃水不忘挖井人的敬意与感恩。布谷鸟哈希的思想和算法由密码学领域的前人提出和设计，将布谷鸟哈希算法和具有强哈希性能的 SHA-2 系列哈希函数结合起来搭建的管理系统，是一种可以在现实场景中应用的管理系统。拥有了前人的探索与成就，我们后人才能登上巨人肩，立足更高，看得更远。

谨在此深切感谢为本论文的完成给予帮助的全体人员。首先，感谢我的指导教师田海博副教授。作为本科生的我，学习经验不足，对学术钻研缺乏一技之长。田老师治学严谨，学识渊博，在我面对毕业课题颇感无助的迷茫时期，为我的生活提供了积极性的建议，为论文的定题做出了方向性指导。在毕业设计处于瓶颈阶段，与导师的交流不但为我带来了实用性的启发，还拓宽了我的知识领域。其次，还要感谢在学习历程遇到困难之际，多次给予热心帮助，互相交流学习心得的同专业室友，感谢协助我调试代码，纠正细小错误的同级同学和师哥师姐。

最后，谢谢一直支持我的家人，谢谢家人们对我始终如一的鼓励。在大学期间，他们尽心尽力帮助我调整心理状况，缓解情绪压力，稳固学习心态，优化学习环境，对我的学习生活无条件支持和付出，得以让我在艰难曲折的道路上砥砺前行。日后，我定不会辜负所有人的期望，以成为家国栋梁之才为目标，矢志不渝，乘风破浪。